



**Lab Report**

|                                                                                    |
|------------------------------------------------------------------------------------|
| <b>Lab Report No</b> : 02                                                          |
| <b>Lab Report Name</b> : Solution of 8-Puzzle Problem using DFS, DLS and IDS       |
| <b>Course Title</b> : Artificial Intelligence Sessional                            |
| <b>Course Code</b> : CSE 422                                                       |
| <b>Name</b> : MD Rakibul Hassan                                                    |
| <b>ID</b> : 0822220105101024                                                       |
| <b>Level</b> : 4 <b>Term</b> : 2 <b>Section</b> : A <b>Group</b> : G1              |
| <b>Date of Submission</b> : 10/05/2026 <b>Semester</b> : Spring <b>Year</b> : 2026 |

**Marking Rubric**

| Problem Understanding & Report Clarity (3) | Implementation (4) | Results, Analysis & Presentation (3) | Total (10) |
|--------------------------------------------|--------------------|--------------------------------------|------------|
|                                            |                    |                                      |            |

**Key Learnings:**

- Learned about the DFS, DLS and IDS algorithms with hands-on experience.
- Understand the working of the usage limit and the repeatedly applying DLS by increasing the depth limit step by step until the goal is not found.

**Code Implementation:**

**Using DFS**

```
# 8-Puzzle Problem using DFS
# =====
# --- Check if state is goal ---
def is_goal(state, goal):
    return state == goal

# --- Find blank position ---
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# --- Generate successors ---
def get_successors(state):
    successors = []
    x, y = find_blank(state)

    # Possible moves: Up, Down, Left, Right
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dx, dy in moves:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]

            # Swap blank tile with neighbor tile
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]

            # Convert list back to tuple
            successors.append(tuple(tuple(row) for row in new_state))
```

**Figure No: 01**



## Using DLS

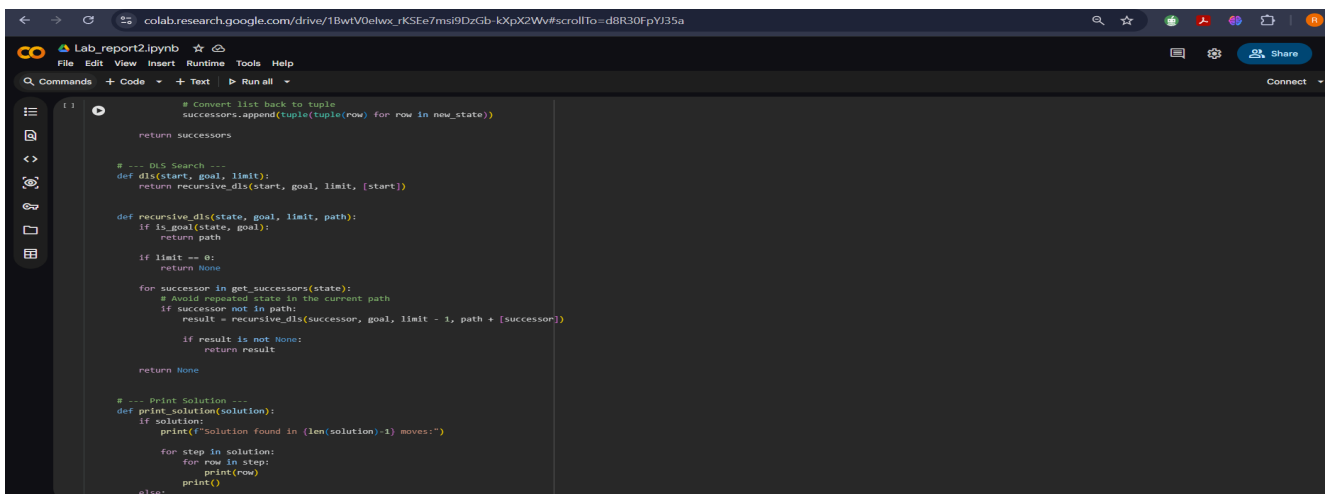


```
def is_goal(state, goal):
    return state == goal

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def get_successors(state):
    successors = []
    x, y = find_blank(state)
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]
            # Swap blank tile with neighbor tile
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            # Convert list back to tuple
```

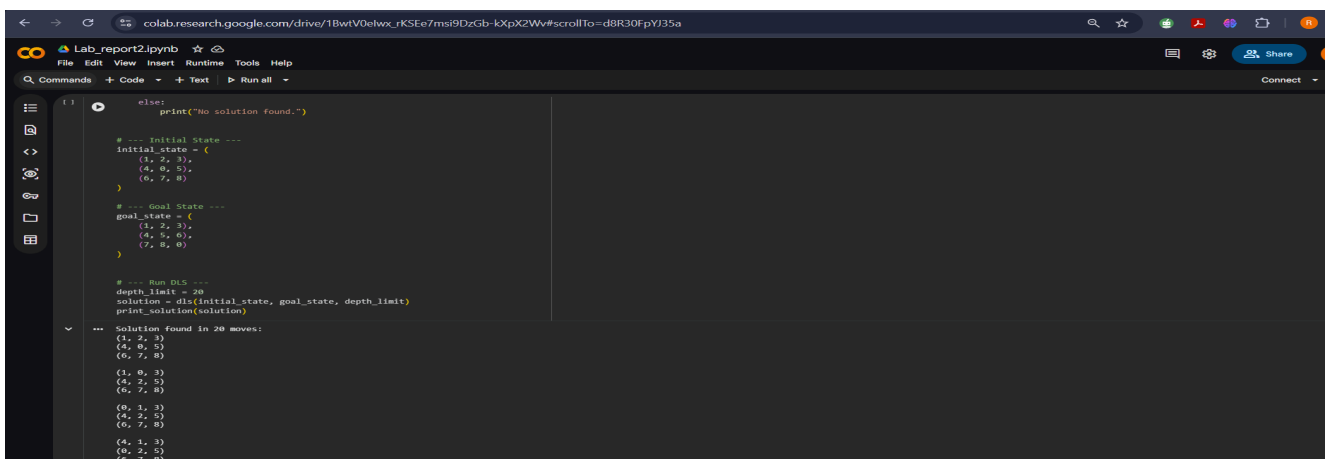
Figure No: 05



```
def recursive_dis(state, goal, limit, path):
    if is_goal(state, goal):
        return path
    if limit == 0:
        return None
    for successor in get_successors(state):
        # Avoid repeated state in the current path
        if successor not in path:
            result = recursive_dis(successor, goal, limit - 1, path + [successor])
            if result is not None:
                return result
    return None

def print_solution(solution):
    if solution:
        print(f'Solution found in {len(solution)-1} moves:')
        for step in solution:
            for row in step:
                print(row)
            print()
```

Figure No: 06



```
def main():
    # Initial State
    initial_state = ((1, 2, 3), (4, 0, 5), (6, 7, 8))
    # Goal State
    goal_state = ((1, 2, 3), (4, 5, 6), (7, 8, 0))
    # Run DLS
    depth_limit = 20
    solution = dls(initial_state, goal_state, depth_limit)
    print_solution(solution)

if __name__ == '__main__':
    main()
```

Solution found in 20 moves:

```
((1, 2, 3),
(4, 0, 5),
(6, 7, 8))
((1, 0, 3),
(4, 2, 5),
(6, 7, 8))
((0, 1, 3),
(4, 2, 5),
(6, 7, 8))
((4, 1, 3),
(0, 2, 5),
(6, 7, 8))
```

Figure No: 07

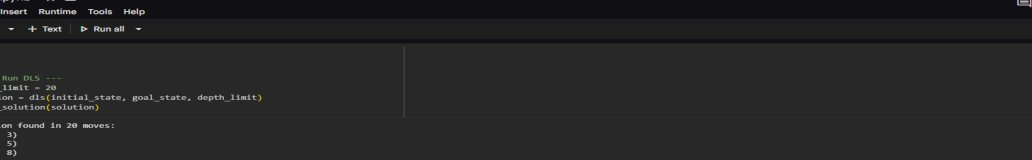
## Sample Input, Output, and Result Analysis:

### Input Sample:

initial\_state = ((1, 2, 3), (4, 0, 5), (6, 7, 8))

goal\_state = ((1, 2, 3), (4, 5, 6), (7, 8, 0))

### Output Sample:



The screenshot shows a Google Colab notebook with the following content:

```
Lab_report2.ipynb
File Edit View Insert Runtime Tools Help
Commands Code Text Run all
Solution found in 20 moves:
(1, 2, 3)
(4, 0, 5)
(6, 7, 8)
(1, 0, 3)
(4, 2, 5)
(6, 7, 8)
(0, 1, 3)
(4, 2, 5)
(6, 7, 8)
(4, 1, 3)
(0, 2, 5)
(6, 7, 8)
(4, 1, 3)
(0, 2, 5)
(6, 7, 8)
(4, 1, 3)
(0, 2, 5)
(6, 7, 8)
(4, 1, 3)
(0, 0, 5)
(7, 2, 8)
(4, 1, 3)
(0, 0, 5)
(7, 2, 8)
(0, 1, 3)
```

**Figure No: 08**

## Using IDS

The screenshot shows a Jupyter Notebook titled "Lab\_report2.ipynb" in a web browser. The notebook contains a Python script for solving an 8-Puzzle problem using ID3. The script is as follows:

```

# 8-Puzzle Problem using ID3
# -----

# --- Check if state is goal ---
def is_goal(state, goal):
    return state == goal

# --- Find blank position ---
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# --- Generate successors ---
def get_successors(state):
    successors = []
    x, y = find_blank(state)

    # Possible moves: Up, Down, Left, Right
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dx, dy in moves:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]

            # Swap blank tile with neighbour tile
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]

            # Convert list back to tuple
            successors.append(tuple(tuple(row) for row in new_state))

```

**Figure No: 09**

```
colab.research.google.com/drive/1BwtV0elwx_rKSEe7msi9DzGb-kXpX2Wv?scrollTo=ZyyA31NqlAiv

Lab_report2.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code - + Text | Run all -

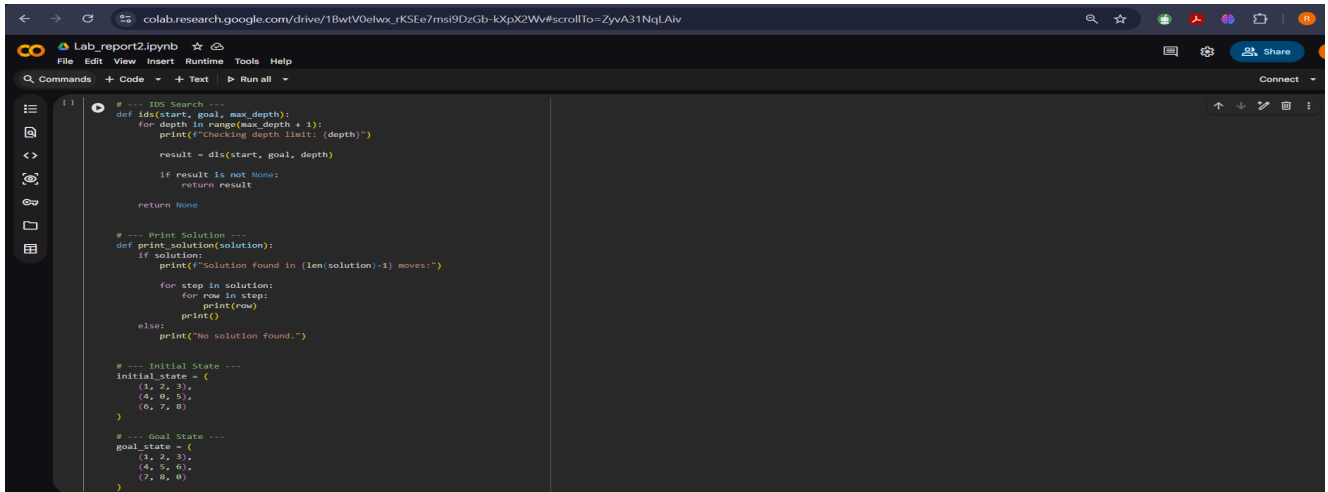
return successors

# --- DFS function for IDS ---
def dfs(start, goal, limit):
    return recursive_dfs(start, goal, limit, [start])

def recursive_dfs(state, goal, limit, path):
    if is_goal(state, goal):
        return path
    if limit == 0:
        return None
    for successor in get_successors(state):
        # Avoid repeated states in the current path
        if successor not in path:
            result = recursive_dfs(successor, goal, limit - 1, path + [successor])
            if result is not None:
                return result
    return None

# --- IDS Search ---
def ids(start, goal, max_depth):
    for depth in range(max_depth + 1):
        print(f"Checking depth limit: {depth}")
        result = dfs(start, goal, depth)
        if result is not None:
            return result
    return None
```

**Figure No: 10**



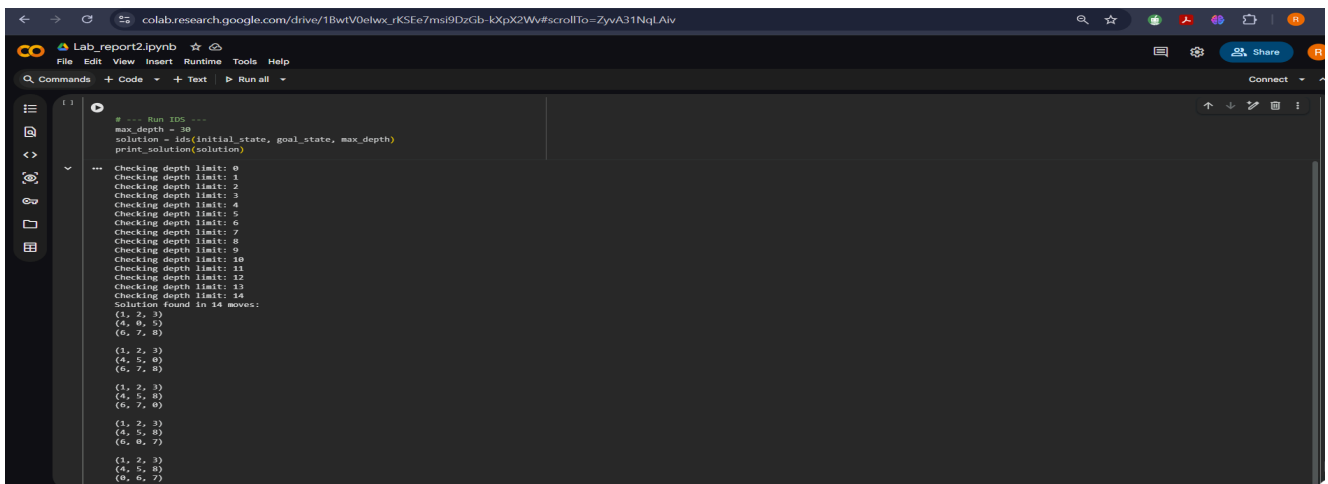
```
def id3(start, goal, max_depth):
    for depth in range(max_depth + 1):
        print(f"Checking depth limit: {depth}")
        result = dfs(start, goal, depth)
        if result is not None:
            return result
    return None

def print_solution(solution):
    if solution:
        print(f"Solution found in {len(solution)-1} moves:")
        for step in solution:
            for row in step:
                print(row)
    else:
        print("No solution found.")

# --- Initial State ---
initial_state = (
    (1, 2, 3),
    (4, 0, 5),
    (6, 7, 8)
)

# --- Goal State ---
goal_state = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)
```

Figure No: 11



```
# --- Run ID3 ---
max_depth = 30
solution = id3(initial_state, goal_state, max_depth)
print_solution(solution)

Checking depth limit: 0
Checking depth limit: 1
Checking depth limit: 2
Checking depth limit: 3
Checking depth limit: 4
Checking depth limit: 5
Checking depth limit: 6
Checking depth limit: 7
Checking depth limit: 8
Checking depth limit: 9
Checking depth limit: 10
Checking depth limit: 11
Checking depth limit: 12
Checking depth limit: 13
Checking depth limit: 14
Solution found in 14 moves:
(1, 2, 3)
(4, 5, 0)
(6, 7, 8)
(1, 2, 3)
(4, 5, 0)
(6, 7, 8)
(1, 2, 3)
(4, 5, 0)
(6, 7, 8)
(1, 2, 3)
(4, 5, 0)
(6, 7, 8)
(1, 2, 3)
(4, 5, 0)
(6, 7, 8)
```

Figure No: 12

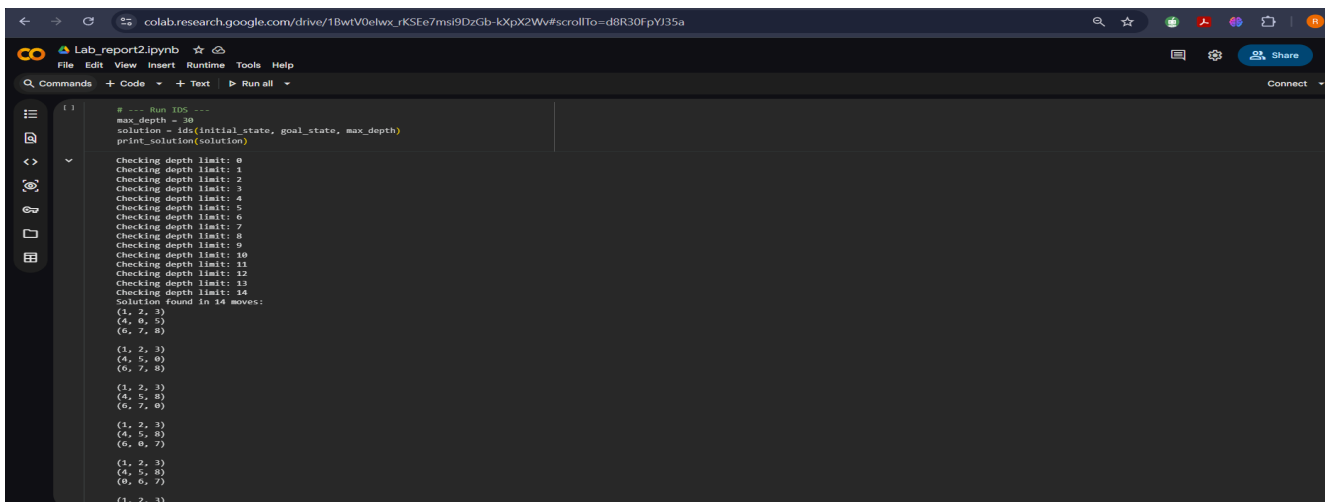
## Sample Input, Output, and Result Analysis:

### Input Sample:

initial\_state = ((1, 2, 3), (4, 0, 5), (6, 7, 8))

goal\_state = ((1, 2, 3), (4, 5, 6), (7, 8, 0))

### Output Sample:



```
# --- Run ID3 ---
max_depth = 30
solution = id3(initial_state, goal_state, max_depth)
print_solution(solution)

Checking depth limit: 0
Checking depth limit: 1
Checking depth limit: 2
Checking depth limit: 3
Checking depth limit: 4
Checking depth limit: 5
Checking depth limit: 6
Checking depth limit: 7
Checking depth limit: 8
Checking depth limit: 9
Checking depth limit: 10
Checking depth limit: 11
Checking depth limit: 12
Checking depth limit: 13
Checking depth limit: 14
Solution found in 14 moves:
(1, 2, 3)
(4, 5, 0)
(6, 7, 8)
(1, 2, 3)
(4, 5, 0)
(6, 7, 8)
(1, 2, 3)
(4, 5, 0)
(6, 7, 8)
(1, 2, 3)
(4, 5, 0)
(6, 7, 8)
(1, 2, 3)
(4, 5, 0)
(6, 7, 8)
```

Figure No: 13

## **Result Analysis**

- **While DFS can solve the problem, BFS offers a more optimal solution for it.**
- **A total of 19,992 moves using DFS, 20 moves using DLS and 14 moves in IDS are needed for solving the 8-puzzle problem.**
- **DFS uses a stack or recursion.**
- **DLS also uses a stack or recursion but stops exploration once it reaches a pre-defined depth limit.**
- **IDS repeatedly uses a stack or recursion to perform DLS at increasingly deeper limits until the goal is found.**